

Setting Up and Running RSQSim

-Evan Marschall, Bruce Shaw and Alice-Agnes Gabriel

Last updated March 2026

(Based of some documentation supplied by Keith Richards-Dinger)

Important Note about citing:

If you are using RSQSim please cite the following 3 papers as well as any software release you may be using.:

1. Dieterich, 1995:

<https://npg.copernicus.org/articles/2/109/1995/npg-2-109-1995.pdf>

2. Dieterich & Richards-Dinger, 2010:

<https://link.springer.com/article/10.1007/s00024-010-0094-0>

3. Richards-Dinger & Dieterich, 2012:

<https://pubs.geoscienceworld.org/ssa/srl/article/83/6/983/315277/RSQSim-Earthquake-Simulator?guestAccessKey=>

Before Compiling (what you need)

C compiler, such as gcc

Based on your c compiler you may need to edit the Makefile

MPI, this can be OpenMPI or Intel MPI

Based on your MPI, you may need to edit the Makefile

Make, you will need this to target the makefile

Based on whether you have mac or windows, you may need to install make.

* If you want to use the old postprocessing scripts you will need an older version of R (R version 4.2.3). However, I have moved away from this and utilize python scripts instead.

Compiling the Source Code

To compile the source code simply execute make within the directory. You may have to adapt the make file based on your specific architecture.

Then run **“make runRSQSim”** in the terminal *This will compile the code only, not the R based post processing stuff. **(IMPORTANT, AS OF RIGHT NOW ONLY THE SOURCE CODE IS UPLOADED. THEREFORE ONLY “make runRSQSim” Will Work.**

*** (Right now this is not included in github: just run “make runRSQSim”.** If you run “make” it will try to compile everything including the post process R procedures. If you would like to use the old R scripts you will need to do a few extra steps (this is for mac, may be different for windows)

- Install R version 4.2.3 or lower
- Open an R terminal, just input “R”
- In R run *"install.packages(“devtools”)"*
- Exit R and in the directory where RSQSim is run “make RSQSimPostProcessT”
- After that run “make .RProfile” so you can call the R functions from anywhere on your machine.

Running the Code and Input parameters

To run the RSQSim code simply execute the following in the terminal:

```
runRSQSim input_file.in
```

Or to run in parallel

```
mpirun -np 8 runRSQSim input_file.in ( np is the number of processes)
```

Where the input file is a file containing a suite of parameters and their values and will be in the form:

Parameter_Name = Parameter_Value (There must be spaces on either side of “ = “

Table 1 and 2 in the next section has a list of the input parameters, their type and default value.

Input Parameters (Input_file.in):

RSQsim uses the a rate and state friction law:

$$\tau = \sigma \left(\mu_0 + a \ln \left(\frac{V}{V^*} \right) + b \ln \left(\frac{\theta V^*}{D_c} \right) \right)$$

With an aging law:

$$\dot{\theta} = 1 - \frac{\theta V}{D_c} - \alpha \frac{\theta \dot{\sigma}}{b \sigma}.$$

parameterName	type	default	units
A_1	float	0.001	m
fA	float	1.0	
B_1	float	0.015	
Dc_1	float	10^{-5}	
mu0_1	float	0.6	m/s
ddotStar_1	float	10^{-6}	
ddotAB_1	float	10^{-6}	m/s
alpha_1	float	0.25	s
theta0_1	float	$2 \cdot 10^8$	
tau0_1	float	60.0	MPa
sigma0_1	float	100.0	MPa
sigmaFracPin	float	0.0	s
maxThetaPin	float	10^{10}	
ddotEQ_1	float	1.0	m/s
ddotEQFname	string		MPa
stressOvershootFactor	float	0.10	
lameLambda	float	10^4	
lameMu	float	10^4	
slowSlip_1	integer	0	s
nEq	integer	1000	
tStart	float	0.0	s
maxT	float	10^{100}	s
faultFname	string	faults.in	
outFnameInfix	string		
writeTau	integer	0	
writeSigma	integer	0	
writeSlip	integer	0	
writeSlipSpeed	integer	0	
writeState	integer	0	
writeTheta	integer	0	
writeTransitions	integer	1	

Table 1: Parameter names, types, and default values

paramterName	type	default	units
minDtWrite	float	0.0	s
minDtWriteCoseismic	float	0.0	s
minDtWriteInterseismic	float	0.0	s
minMagWrite	float	0.0	
writeStiffness	integer	0	
stressRateSpecification	integer	1	
dMu3	float	0.001	
initTauFname	string		
initSigmaFname	string		
initThetaFname	string		
initSlipSpeedFname	string		
AFname	string		
BFname	string		
DcFname	string		
mu0Fname	string		
ddotStarFname	string		
ddotABFname	string		
alphaFname	string		
KTauFname	string		
KSigmaFname	string		
tFailFname	string		
tauFailFname	string		
tauDotFname	string		
sigmaDotFname	string		
pinnedFname	string		
neighborFname	string		
stressRateFanme	string		
slowSlipFname	string		
DEBUG	integer	0	

Table 2: More parameter names, types, and default values

A_1: The rate coefficient a in the rate and state friction law. This value will be used for all fault elements unless a separate **AFname** is provided.

fA: The value that a is multiplied by during nucleation

B_1: The state coefficient b in the rate and state friction law. This value will be used for all fault elements unless a separate **BFname** is provided.

Dc_1: The characteristic slip distance (m) in the rate and state friction law and the aging law. This value will be used for all fault elements unless a separate **DcFname** is provided.

Mu0_1: μ_0 in the rate and state friction law. This is used for all elements unless a separate **mu0Fname** is provided.

ddotStar_1: This is the reference slip velocity V^* in the rate and state friction law. This value will be used for all fault elements unless a separate **ddotStarFname** is provided.

ddotAB_1: This is only really used for slow slip events, it is the slip speed for a slip rate transition on slow slip patches.

alpha_1: The coefficient α which relates changes in the normal stress to changes in the state variable in the aging law. This value will be used for all fault elements unless a separate **alphaFname** is provided.

theta0_1: The initial value of the state variable θ in the rate and state friction law (units of seconds). This value will be used for all fault elements unless a separate **initThetaFname** is provided.

tau0_1: The initial value of the shear stress τ in the rate and state friction law (units of MPa). This value will be used for all fault elements unless a separate **initTauFname** is provided.

sigma0_1: The initial value of the normal stress σ in the rate and state friction law (units of MPa). This value will be used for all fault elements unless a separate **initSigmaFname** is provided.

sigmaFracPin: If the normal stress on any patch drops to the product of sigmaFracPin and that patch's initial normal stress, that patch is locked for the remainder of the simulation.

maxThetaPin: If the state variable on any patch ever exceeds maxThetaPin + 1000t, that patch is locked for the remainder of the simulation.

ddotEQ_1: The slip speed during state 2 (seismic slip) V^{EQ} is given by ddotEQ_1 (m/s). Value is to be used for all fault elements unless **ddotEQFname** is provided

For all “value”Fname inputs: These should all be single column ascii files where the *ith* line corresponds to the parameter value for the *ith* fault patch. See the short example below of what the contents of a Fname file could look like (**AFname**):

```
0.01
0.008
0.01
0.01
0.008
```

In this 5 line example the a coefficient for the rate and state friction law for 5 fault patches. In this example the first patch has an a value of 0.01, the second patch has an a value of 0.008, the third patch has an a value of 0.01, the fourth patch has an a value of 0.01, the fifth patch has an a value of 0.008.

stressOvershootFactor State 2 (seismic slip) nominally ends when the shear stress drops to the “steady-state” stress (which is in quotes because when normal stress is time-varying the concept of steady-state is not quite valid) at the seismic slip speed V^{EQ} . Actually it ends when the stress drops somewhat below this “steady-state” stress with the extra stress drop given as stressOvershootFactor times the stress drop from the peak stress (upon entering state 2) to the “steady-state” stress.

lameLambda and lameMu: The Lamé elastic moduli λ and μ (in MPa).

slowSlip_1: slowSlip 1 should be 0 if all the patches are not slow slip patches and 1 if they all are. Ignored if a value is supplied for **slowSlipFname**. Any patches designated as slow slip patches will use an alternate constitutive law with a low-state cutoff:

$$\tau = \sigma \left(\mu_0 + a \ln \left(\frac{V}{V^*} \right) + b \ln \left(\frac{\theta V^*}{D_c} + 1 \right) \right)$$

nEq and maxT: nEq is the number of events wanted and maxT is the maximum simulation time desired. RSQSim will run until nEq events have occurred or maxT seconds of simulated time have passed, **whichever comes first**. Oftentimes I will set one of the values which I want and make the other extremely large 1e100.

tStart: Upon startup, the time in the simulation is set to tStart. If you are continuing from an old run which crashed, you can change this to the appropriate time.

faultFname: This is the fault file which contains all the patches of the fault (mesh), I usually call this something like *filename.flt*. This is a multi column ascii file with whitespace separators. The fault file can either take quad elements or triangular elements, each of which will have a different format: Note that in RSQSim the free surface is the x-y plane and the Z axis points up, therefore all depths (z) should be negative.

Quad Elements: Columns stated below and separated by a single whitespace

x y z l w strike dip rake sliprate section_number section_name

x, y, z: The coordinates of the center of the patch (in meters)

l, w: The along-strike and down-dip dimensions of the fault patch (in meters)

strike, dip: The orientation of the fault patch (in degrees and using the convention

of Aki and Richards, 2002)

rake: The direction of motion of the hanging wall relative to the footwall (in degrees and using the convention of Aki and Richards, 2002)

sliprate: The long-term average slip rate (**in m/s**) if **stressRateSpecification** is 1. If **stressRateSpecification** is 2 or 3, then this 9th column is ignored but must still be present. **stressRateSpecification** is explained later in the document.

section_number, *section_name*: The section number that the patch belongs to and the name of that section. The *section_name* **should not contain any whitespace and should not be able to be interpreted as a number.**

Triangular Elements: Columns stated below and separated by a single whitespace.

x1 y1 z1 x2 y2 z2 x3 y3 z3 rake sliprate section_number section_name

In this case the *x1 y1 z1 x2 y2 z2 x3 y3 z3* give the coordinates of the three vertices of the triangular elements. These vertices should be ordered such that the right hand rule would give an upward facing normal vector for dipping faults.

writeTau, writeSigma, writeSlip, writeSlipSpeed, writeState, and writeTheta

The values of these parameters determine how often the respective quantities are written out into a binary file. Possible values are:

0: never

1: at every transition

2: before and after every event

4: at every transition during an event

Any sum of the last three write out multiple files according to which of the last three went into the sum. The files will be named something like

tau.outFnameInfix.out.writeTau (where *writeTau* will be 1, 2, or 4. For example, if the parameter *writeTau* is 3, then two shear stress output files will be written

tau.outFnameInfix.out.1 and tau.outFnameInfix.out.2) Note that how often things get written out may be modified by **minDtWrite** and **minMagWrite** (which are both explained later).

writePED

If writePED is non-zero, then five files named

eqs.outFnameInfix.pList

eqs.outFnameInfix.eList

eqs.outFnameInfix.dList

eqs.outFnameInfix.tList

eqs.outFnameInfix.dtauList

are written out.

eqs.outFnameInfix.pList: patch number for each patch that slipped during an event

eqs.outFnameInfix.eList: the event number

eqs.outFnameInfix.dList: the distance slipped during the event for each patch

eqs.outFnameInfix.tList: the time of first rupture for that patch during that event

eqs.outFnameInfix.dtauList: stress drop for each patch during an event

*These files are index 1 based and binary (Some examples of dealing with these files can be found in the *Post_Processing* Jupyter Notebook).

writeTransitions: If this is non-zero it will write a binary file that has three values for the time, element and state in which the transition occurred. For example an element might transition from state 0 to state 1 at $t = 100$ s and you want to catalogue this. The written file will be named something like *trans.outFnameInfix.out*.

minDtWrite: Write out tau, sigma, theta, slip, slip speed and/or state at most every minDtWrite (seconds). Affects both *.1 and *.4 output files.

minDtWriteCoseismic and minDtWriteInterseismic: Allows separate values to be given for minDtWrite during and between ruptures, respectively. minDtWriteCoseismic will affect *.1 and *.4 output files, while

minDtWriteInterseismic will affect only *.1 output files. If these are not given and minDtWrite is, minDtWrite will be used for both. If these are given, any value given for minDtWrite will be ignored.

minMagWrite

Only write out tau, sigma, theta, slip, slip speed, and/or state for events with magnitude $\geq$ minMagWrite. Affects *.2 and *.4 output file.

writeStiffness: Which stiffness matrices to write out. Possible values are:

0 none

1 $K\tau$

2 $K\sigma$

3 $K\tau$ and $K\sigma$

The output files will be named like Ktau.outFnameInfix.out and Ksigma.outFnameInfix.out. They will be ascii files with np lines and each line with np whitespace separated entries, where np is the number of patches in the fault model, and the jth field on the ith line is K_{ij} (in MPa/m). **These can be quite large so I recommend not writing them out unless necessary.**

stressRateSpecification: The value of stressRateSpecification indicates how the tectonic stressing rates are to be determined for each patch. Possible values are:

1: The 9th column of faultFname is interpreted as a long-term average slip rate (in m/s), and the stressing rates (both shear and normal) are determined via a backslip calculation.

2: Stressing rates (in MPa/s) are provided in **tauDotFname** and/or **sigmaDotFname**. If one (or both) of these file names is not supplied, the corresponding stressing rates are set to zero.

3: A constant stressing rate tensor is provided in **stressRateFname** and this rate tensor is projected onto each fault patch to give the shear and normal stressing rates.

dMu3: Any patches for which $a > b$ (and are therefore rate strengthening) are assigned a state of 3 (which never changes) and are approximated as always being in steady-state. As such, their slip speed is uniquely determined by the stresses acting on them. This continuous variation of slip speed is approximated by a piece-wise constant function of time: the speed is only updated when μ (the ratio of shear to normal stress τ/σ) deviates by an amount $d\mu_3$ from that μ which was used to set the current slip speed.

tauDotFname, sigmaDotFname:

If stressRateSpecification is 2, then the stressing rates for each patch are read from these files. Each should be a single-column ascii file, with each line giving the stressing rate (in MPa/s) for the corresponding patch in **faultFname**. If one (or both!) of these files are not supplied, the corresponding stressing rates are set to zero.

pinnedFname

If a value is supplied for this then some of the patches may be pinned (not allowed to slip). The file whose name is supplied should be a single-column numeric ascii file where if the i th line is non-zero then the i th patch is not allowed to slip.

neighborFname: In RSQsim there isn't a well resolved stress concentration at the rupture front. To address this limitation adjacent neighbor patches to the patch that is slipping are "weakened" by reducing the a coefficient. If nothing is supplied for neighborFname then a is reduced on all state 1 patches during rupture. It is better to supply a value for **neighborFname** which should be an ascii file with one line per fault patch. Each line should be a white-space separated list of the patch numbers of the neighbors of the corresponding fault patch. Note that these patch numbers are 0-based.

Output files

The only file that is always output is one containing basic "catalog" information. It is named eqs.outFnameInfix.out. It is an ascii file. The first line gives the revision number of the code that was used in the run. Then there are a number of lines giving all the control parameters used (in the same format that is used for **input_file.in**). Then follows one line for each earthquake with nine columns:

t0, M0, Mw, x, y, z, area, dt, nuc

Where:

t0: origin time in seconds

M0: seismic moment in N-m

Mw: moment magnitude ($M_w = (2/3)(\log_{10}(M_0) - 9.1)$)

x, y, z: hypocentral coordinates in meters

area: total area which ruptured in m²

dt: duration in seconds

nuc: the element that nucleated the rupture

Example steps of setting up your first simulation.

1. Make a fault file: faultFname.flr

The first step is to construct a fault file in the format the RSQSim expects (see **faultFname** section). I find it useful to use gmsh to make a triangular mesh and then convert that mesh file into the RSQSim format. (There is an example of how to do this in the `step_over_mesh` subdirectory in the main repository).

2. Make your input_file.in parameter file

The second is to make the `input_file.in` which contains all the initial parameters. I suggest for an initial run to just start with uniform conditions, so assigning values but keeping all the frame lines empty. See example below (This is for the very short run that is used in the jupyter notebook examples):

```

A_1 = 0.010000000000000000208
fA = 0.100000000000000000555
B_1 = 0.014999999999999999445
Dc_1 = 1.0000000000000000818e-05
mu0_1 = 0.5999999999999999778
ddotStar_1 = 9.999999999999995475e-07
ddotAB_1 = 9.999999999999995475e-07
alpha_1 = 0.25
theta0_1 = 200000000
tau0_1 = 60
sigma0_1 = 100
sigmaFracPin = 0.1
maxThetaPin = 1000000000
ddotEQ_1 = 1
ddotEQFname =
stressOvershootFactor = 0.100000000000000000555
lamelambda = 30000
lameMu = 30000
slowSlip_1 = 0
nEq = 1e10
tStart = 0
maxT = 3.5e11
faultFname = Step_over.flt
outFnameInfix = step_over_test
writeTau = 2
writeSigma = 2
writeSlip = 2
writeSlipSpeed = 2
writeState = 2
writeTheta = 2
writePED = 1
writeTransitions = 1
minDtWrite = 3.15e10
minDtWriteCoseismic = 0
minDtWriteInterseismic = 0
minMagWrite = 6
writeStiffness = 0
stressRateSpecification = 1
dMu3 = 0.0010000000000000000208
initTauFname =
initSigmaFname =
initThetaFname =
initSlipSpeedFname =
AFname =
BFname =
DcFname =
mu0Fname =
ddotStarFname =
ddotABFname =
alphaFname =
KTauFname =
KSigmaFname =
tFailFname =
tauFailFname =
tauDotFname =
sigmaDotFname =
pinnedFname =
neighborFname =
stressRateFname =
slowSlipFname =
writePatchFname =
DEBUG = 1
ZBrentUpperBracket = 0
ZBrentTol = 9.999999999999995475e-07
lowSigmaAction = 0
KZeroFrac = 0.0
KZeroFname =
slipInState1 = 1
resetStressingRates = 1

```

3. Run RSQSim (runRSQSim input_file.in)

4. Plotting Results

After running you should look at the jupyter notebook to see how to do simple post processing and plotting of the results.